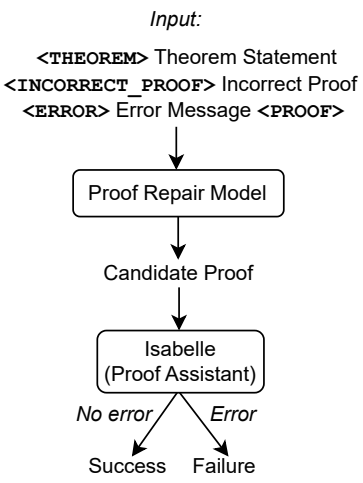

Figure 2: An example of using the proof repair model to repair an incorrect proof.

Building off our proof generation approach, we explore the use of error messages to improve neural theorem provers by developing a proof repair approach. Starting with just the problem statement, we apply the proof generation model from Section 2.1 to sample a proof attempt. If Isabelle accepts the proof attempt, we can stop. Otherwise, we use the error message returned by the proof checker and the incorrect proof attempt to construct an example to serve as input to the *proof repair model*. As depicted in Figure 2, we use the theorem statement, the incorrect proof, and the error message as input to our proof repair model. We then sample the proof attempt from this model, and perform proof checking in the same way as the proof generation approach. Explicitly, the input and output of our proof repair approach pipeline are as follows:

- **Input:** theorem statement, incorrect proof, error message.
- **Output:** candidate proof.

Example. Starting from the theorem `fun_sum_commute`, we illustrate an example of the proof repair approach in our tool Baldur. We apply the proof generation model to obtain more proof attempts. The following is a proof attempt generated by Baldur, which fails in the proof checker.

```
proof (induct A)
case (insert x A)
thus ?case
  by (simp add: assms(2))
qed simp
```

Baldur attempts to apply an induction, but fails to first break down the proof into two cases (finite vs. infinite set). Isabelle returns the following error message:

```
Step error: Unable to figure out induct rule
At command "proof" (line 1)
```

The error message details where the error occurs (line 1) and that the issue is regarding the induct rule. With these strings as input, using the proof repair model, Baldur can attempt to generate a correct proof for this statement. If we want to instead derive a proof repair training example from these strings, we concatenate the theorem statement, the failed proof attempt, and the error message to serve as the input, and we use the correct human-written proof (recall from previous section) as the target.

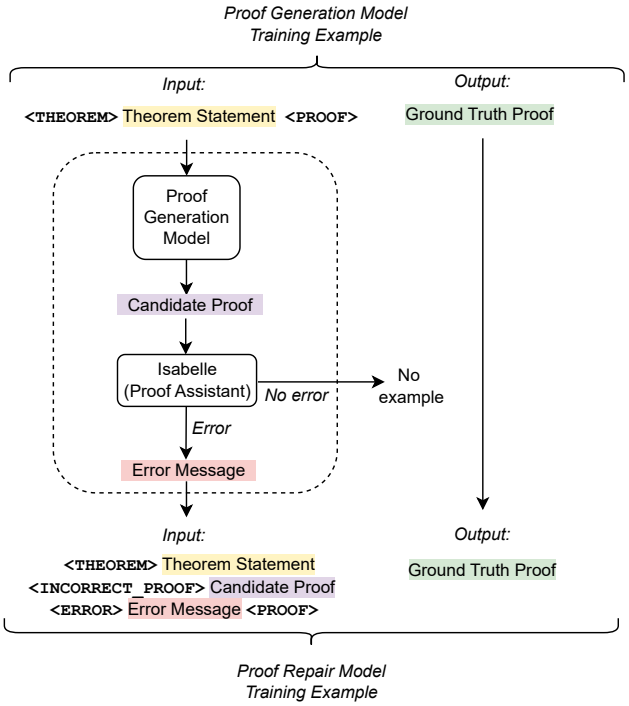

Figure 3: Training data creation for the proof repair model.

Training Data Creation. To train the proof repair model, we need to generate a proof repair training set. Figure 3 details the training data creation process. Using the proof generation model, we sample one proof with temperature 0 for each problem in the original training set used to train the proof generation model. Using the proof assistant, we record all failed proofs and their error messages. We then proceed to construct the new proof repair training set. For each original training example, we concatenate the theorem statement, the (incorrect) candidate proof generated by the proof generation model, and the corresponding error message to obtain the input sequence of the new training example. For the target sequence, we reuse the ground truth proof from the original training example. We fine-tune the pretrained LLM on the proof repair training set to obtain the proof repair model.

2.3 Adding Context

LLMs possess impressive in-context learning abilities (cf. [7, 14]) that allow them to flexibly use information that is provided as part of the input sequence (and, in fact, as part of their own output [61, 88]). In order to explore to what extent in-context learning can help in the theorem proving domain, we extend their inputs with potentially helpful context. Adding to our proof generation approach, we use the theory file contexts (the lines preceding the theorem statement)

as input to our *proof generation model with context*. Explicitly, the input and output of our proof generation model with context is as follows:

- **Input:** theory file context and theorem statement.
- **Output:** candidate proof.

Example. Continuing the example, the theory file context directly preceding `fun_sum_commute` is the following theorem statement and its associated proof.

```
lemma additive_implies_homogenous:
  assumes "⋀x y. f (x + y) = f x +
  ((f (y::'a::monoid_add))::'b::cancel_comm_monoid_add)"
  shows "f 0 = 0"
proof -
  have "f (0 + 0) = f 0 + f 0" by (rule assms)
  hence "f 0 = f 0 + f 0" by simp
  thus "f 0 = 0" by simp
qed
```

The proof generation model with context in Baldur can leverage this additional information. Strings that appear in the theorem statement for `fun_sum_commute`, such as `"f 0 = 0"`, appear again in this context, and so the additional information surrounding them could help the model make better predictions.

Training Data Creation. We add the lines of the theory file that precede the theorem statement to serve as additional context. This means that context can include statements, such as the previous theorems, definitions, proofs, and even natural language comments. To make use of the available input length of LLMs, we first add up to 50 preceding statements from the same theory file. During training, we first tokenize all these statements, and then we truncate the left of the sequence to fit the input length.

Premise Selection. Many proofs make frequent use of definitions and previously proven statements, also known as *premises*. Some neural theorem provers, such as HOList [4], focus entirely on the problem of selecting the right set of premises, which has been shown to be quite successful in theorem proving.

Premise selection is clearly similar to the addition of context in some aspects, but we want to emphasize some key differences: (1) Adding context is an extremely simple technique that only requires rudimentary text processing, (2) by adding the preceding lines of the theory file, the model can only observe a small fraction of the available premises, (3) most of the added context consists of proofs.

2.4 Large Language Model

We use Minerva [48], a large language model pretrained on a mathematics corpus based on the PaLM [14] large language model. Specifically, we use the 8 billion parameter model and the 62 billion parameter model. The Minerva architecture follows the original Transformer architecture [85], but has some noteworthy differences. It is a decoder-only transformer with maximum sequence length of 2,048 tokens. The model uses

- rotary position encodings [79] instead of sinusoidal absolute position embeddings,
- parallel layers [6], which compute the feed forward layer and the attention layer in parallel and add up their results instead of computing them in sequence, and
- multi-query attention, which uses a single key-value pair per token per layer for faster decoding [77].

As this model is not a contribution of this paper, we refer the reader to prior work for lower-level details on the Minerva architecture [14].

Baldur-specific implementation details. The proof generation task naturally consists of an input, which is the theorem statement (potentially augmented with additional information), and the output (target), which is the proof for the theorem. To work with the decoder-only model, we concatenate the inputs and targets, but the loss is only computed over the target during fine-tuning. The inputs use bidirectional attention while the targets use causal attention as in PrefixLM [69].

As the transformer has a maximum context length of 2048, we pad the sequences with zeros if they are too short, and we need to truncate them if they are too long. Inputs to the model are truncated to the maximum input length by dropping tokens on the left. The rationale for dropping tokens on the left is that the additional context is given before the theorem statement, and can be truncated more safely than the theorem statement itself. Similarly, targets (i.e. the proof to generate) are truncated on the right to the maximum target length.

We used a maximum input length of 1536 and a maximum target length of 512 all experiments but the repair study, which used 1024 and 1024 instead. We use a drop-out rate of 0.1 for both generation and repair models to address overfitting.

During sampling from the language model we restrict the choice of the next token to the 40 tokens with the highest score, also called top-K sampling [19]. We sample sequences with a maximal length of 256 tokens. The model was trained to generate up to 512 tokens, but since most successful proofs are relatively short, this limitation has little impact on the proof rate while saving some compute.

We use a batch size of 32, and fine-tune for up to 100,000 steps, but we observed that the model begins to overfit to the training set after 50,000 to 70,000 steps. For inference, we selected checkpoints from just before the model started to overfit.

3 EVALUATION

In this section we present several experiments and discuss the following research questions:

RQ1: How effective are LLMs at generating whole proofs?
RQ2: Can LLMs be used to repair proofs?
RQ3: Can LLMs benefit from using the context of the theorem?
RQ4: Does the size of the LLM affect proof synthesis effectiveness?
RQ5: How do LLMs compare to other SOTA proof generation methods?

To answer these questions, we trained several language models using the approach from Section 2, and evaluated them on the PISA benchmark (see Section 3.2). Our main results can be found in Table 4 and in Figure 5.

Model	16 samples	64 samples
Baldur 8b generate	34.8%	40.7%
Baldur 8b generate + repair	36.3%*	—
Baldur 8b w/ context	40.9%	47.5%
Baldur 62b w/ context	42.2%	47.9%
Baldur 8b w/ context ∪ Thor	—	65.7%

Table 4: Proof rate of different models.
***The repair approach uses half the number of samples, and then one repair attempt for each sample.**

3.1 Experimental Setup

Machine specification. For most of the training runs of the 8b model, we used 64 TPUv3 cores distributed across 8 hosts. For training the 62b model, we used 256 TPUv3 cores distributed across 32 hosts. For most inference jobs, we used between 32 inference servers using 8 TPUv3 cores each.

Proof Checker. We use the PISA codebase [34] under a BSD 3-clause license, which allows us to interact with the Isabelle proof assistant to check proofs. To run large jobs of the proof checker, we package it in a Docker container and run it on GCP. We extended the proof checker to discard any proofs that contain "sorry" or "oops", which are keywords that skip proofs, but otherwise pass the proof checker. We apply a timeout of 10 seconds to each proof step in the proof checker.

3.2 PISA Benchmark

We derive our datasets from the PISA dataset [34], which includes the Isabelle/HOL repository under a BSD-style license and the Archive of Formal Proofs (AFP) from October 2021. The AFP is a large collection of Isabelle/HOL proof developments. PISA includes the core higher-order logic library of Isabelle, as well as a diverse library of proofs formalised with Isabelle. This includes mathematics proofs and verification of software and hardware systems. The PISA dataset comes with a 95%/1%/4% split of theorems for the training/validation/test sets, which we follow in this work as well.

For the test set, prior work randomly chose 3,000 theorems from the test set to report their results on. We report our results on the complete test set. Some entries in the dataset are not proper theorems (starting with the keyword "lemmas" instead of "lemma"), which we filter out, as did prior work. This leaves us with a total of 6,336 theorems in our test set (originally 6,633 theorems).

It is worth noting that, as with any LLM-based work, there is the potential for proofs from the test set to have leaked into the LLM pretraining data. While the pretraining data for the Minerva LLM at the base of our models does not include the PISA dataset, it does contain code that may include some Isabelle/HOL proofs found in PISA. This should be kept in mind when interpreting the results.

3.3 RQ1: How effective are LLMs at generating whole proofs?

We aligned our methodology with the methodology described in Thor [33] to enable a comparison between various methods. The Thor paper includes informative baselines for the PISA benchmark, including Sledgehammer, a method relying on heuristic search, and a language model approach using search.

Sledgehammer and the search-based language model approach achieve 25.6% and 39.0%, respectively. In comparison, our naive proof generation approach with an 8b language model achieves a proof rate of 34.8% with 16 samples and of 40.7% with 64 samples. The comparison is even more favorable, if we consider the other variants of Baldur, which achieve a proof rate of up to 47.9%.

We observe that the comparison depends on the computational cost that we spend during inference. While comparing the cost required for the two methods is involved, one measure we can use is the amount of computational resources reserved during proof generation. For a single proof, the language model approach using search [33] requires a TPUv3 with 8 cores for 216 seconds,[2] while our methodology also requires a TPUv3 with 8 cores for around 35 seconds to sample 64 proofs – a difference of factor 6. This argument disregards the time spent on proof checking, which is intentional: proof checking is done on CPUs, which is cheap compared to time spent on TPUs. So, disentangling these two workloads can lead to significant reductions in computational cost.

> **RA1**: These results demonstrate that LLMs can generate full proofs just as well as smaller language models augmented with a search strategy.

3.4 RQ2: Can LLMs be used to repair proofs?

We trained models for proof generation and repair as detailed in Section 2. If we sample from the proof generation model once with temperature 0, collect the failed proofs, and then repair once with temperature 0, we generate an additional 266 or 4.2% correct proofs. However, in this comparison, the generate + repair approach uses two samples, while the generate approach has only one sample. For a fair comparison, we have to compare the repair approach to the generate approach with additional inference attempts.

In Figure 5, we plot the proof success rate of the generate approach and the repair approach against the number of proof attempts. Note that the number of samples for the repair approach does not perfectly align with the number of samples for the generate approach. This is because the generate approach tends to produce multiple copies of the same proofs, which we deduplicate before repair, and only generate one repair attempt per failed proof attempt. For each of the number of samples of the generate approach, we tune the temperature in the range of 0.0 to 1.4 in increments of 0.2, and we always use temperature 0 for the repair approach.

We observe that the repair approach consistently outperforms the plain proof generation model, which only uses the theorem statement as input. However, this does not yet answer the question of where those gains from. To shed some light on this question, we trained another repair model that is given the same information, except that it does not see the error message. Plotting the proof success rate of this model in Figure 5 shows us that while it is able to prove additional theorems, it does not surpass the performance of the generate model when normalized for inference cost. This

[2] Section 4.1 in [33] states that 1000 problems take around 60 TPU hours.